Task 1.3: Hooke–Jeeves and GPS methods

Objective

In this assignment you will implement and test two classic derivative-free search algorithms:

(a) The *Hooke–Jeeves method* (exploratory search with step reduction).

(b) The *Generalized Pattern Search (GPS)* method with a direction set.

The goal is to understand how direct search methods explore the domain, how step-size adaptation influences convergence, and how the choice of directions affects efficiency.

Algorithms to Implement

You must translate the pseudocode below into Python.

Hooke–Jeeves

Algorithm 1 Hooke–Jeeves

```
function HOOKE_JEEVES(f, x, α, ϵ, γ = 0.5)
    y = f(x), n = length(x)
    while α > ϵ do
        improved = false
        x_best = x, y_best = y
        for i = 1 to n do
            for sgn in −1, +1 do
                x′ = x + sgn · α · basis(i, n)
                y′ = f(x′)
                if y′ < y_best then
                    x_best = x′, y_best = y′, improved = true
                end if
            end for
        end for
        x = x_best, y = y_best
        if not improved then
            α = α · γ
        end if
    end while
    return x
end function
```

Generalized Pattern Search (GPS)

Algorithm 2 Generalized Pattern Search (GPS)

```
function GPS(f, x, α, D, ϵ, γ = 0.5)
    y = f(x)
    while α > ϵ do
        improved = false
        for (i, d) in enumerate(D) do
            x′ = x + αd
            y′ = f(x′)
            if y′ < y then
                x = x′, y = y′, improved = true
                D = [d] + D[: i] + D[i + 1 :]          ▷ Promote successful direction
                break
            end if
        end for
        if not improved then
            α = α · γ
        end if
    end while
    return x
end function
```

Test Functions

You will apply both algorithms to a set of benchmark functions:

Test functions (explicit formulas and known minima)

- **Ackley function** (2D):

$$f(x,y) = -20\exp\Big(-0.2\sqrt{\tfrac{1}{2}(x^2+y^2)}\Big) - \exp\Big(\tfrac{1}{2}(\cos(2\pi x)+\cos(2\pi y))\Big) + e + 20.$$

  (Global minimum at $(0,0)$ with $f(0,0)=0$.)

- **Booth function** (2D):

$$f(x,y) = (x+2y-7)^2 + (2x+y-5)^2.$$

  (Global minimum at $(1,3)$ with $f(1,3)=0$.)

- **Branin function** (2D):

$$f(x,y) = \Big(y - \tfrac{5.1}{4\pi^2}x^2 + \tfrac{5}{\pi}x - 6\Big)^2 + 10\Big(1 - \tfrac{1}{8\pi}\Big)\cos(x) + 10.$$

  (Multiple global minima; one example at $(\pi,\, 2.275)$ with $f(\pi, 2.275) \approx 0.397887$.)

- **Rosenbrock function** (2D):

$$f(x,y) = (1-x)^2 + 5(y-x^2)^2.$$

  (Global minimum at $(1,1)$ with $f(1,1)=0$.)

- **Wheeler function** (2D):

$$f(x,y) = -\exp\big(-(xy-1.5)^2 - (y-1.5)^2\big).$$

  (Global minimum at $(1.5,1.5)$ with $f(1.5,1.5)=-1$.)

- **Rastrigin function** (2D):

$$f(x,y) = 10\cdot 2 + (x^2 - 10\cos(2\pi x)) + (y^2 - 10\cos(2\pi y)).$$

  (Global minimum at $(0,0)$ with $f(0,0)=0$.)

Starting Points

Use the following initial points (deterministic runs, no randomness):

- Ackley: $x^{(0)} = (-3,-3)$
- Booth: $x^{(0)} = (0,0)$
- Branin: $x^{(0)} = (2,2)$
- Rosenbrock: $x^{(0)} = (-1.5, 2.0)$
- Wheeler: $x^{(0)} = (1.5, 0.5)$
- Rastrigin: $x^{(0)} = (2.5, 2.5)$

Tasks

1. Implement Hooke–Jeeves and GPS in Python.
2. Run experiments on the listed test functions.
3. Record for each run: final solution x_{found}, $f(x_{found})$, number of iterations, time of working.
4. Produce contour plots with optimization trajectories.
5. Summarize in a results table and discuss:
   - Which method converged faster?
   - Did both methods find the global minimum?
   - How sensitive are they to the starting point?
   - How does step-size reduction affect the performance?

Deliverables

- Python code implementing Hooke–Jeeves and GPS.
- Plots showing trajectories on each functions (e.g. Rosenbrock and Ackley).
- Console table with results: `function`, `start`, `method`, x_found, `f(`x_found`)`, `iterations`, `time of working`.
- Short discussion (3–5 sentences) answering the above questions.

Additional exercises (you have to do one task of your choice)

1. **Adaptive GPS:** Implement GPS with an adaptive set of directions — upon a successful step, generate new orthogonal directions (e.g., via Gram–Schmidt) and compare performance.
2. **Noisy objective:** Add additive Gaussian noise to function evaluations and estimate the result statistics (mean/std) over 20 repeated runs.
3. **Hybrid method:** Combine Hooke–Jeeves and Powell: first run several cycles of HJ, then switch to Powell; evaluate effectiveness on Rosenbrock and Rastrigin functions.
4. **Automatic parameter tuning:** Implement a grid search for parameters alpha0 and gamma (log-steps) and output a sensitivity heatmap.